Name:- Rushikesh Jadhav
Batch:- NGA-MS-Dynamics

Assignment

Azure Cosmos DB Hands-On Lab Report

Introduction

In this lab, we're going to work with Azure Cosmos DB, which is Microsoft's globally distributed, multi-model database service. This is different from the SQL database we worked with earlier - Cosmos DB is a NoSQL database that's designed for handling massive amounts of data with low latency. The cool thing about Cosmos DB is that it can scale globally and handle different types of data models. For this lab, we'll be using the NoSQL API, which stores data as JSON documents rather than traditional tables with rows and columns.

What We're Trying to Accomplish

The goal here is to understand how to set up and use Cosmos DB from start to finish. We need to create a Cosmos DB account, which is the top-level container for everything. Then we'll create a database within that account, and a container within that database to actually store our data. Finally, we'll add some documents (which are just JSON objects) and learn how to query them. This will give us hands-on experience with NoSQL databases and show us how they're different from traditional relational databases like SQL Server.

Step 1: Starting to Create the Cosmos DB Account

We begin by logging into the Azure portal at https://portal.azure.com. From either the Azure portal menu or the home page, we select "Create a resource". This takes us to the Azure Marketplace where we can create all kinds of different resources.

We search for "Azure Cosmos DB" in the search box and hit enter. When the results appear, we select "Create > Azure Cosmos DB". This is where things get interesting because Cosmos DB supports multiple different APIs - we can use it as a document database, a graph database, a table database, or even make it compatible with MongoDB or Cassandra. For this lab, we're going to use the NoSQL API, which is Cosmos DB's native API.

On the "Create an Azure Cosmos DB account" page, we select the "Create" option within the "Azure Cosmos DB for NoSQL" section. This tells Azure we want to use the document-based NoSQL model rather than one of the other options.

Step 2: Configuring Basic Settings

Now we're on the main creation page where we need to configure all the basic settings for our Cosmos DB account. Let's go through each setting and understand what it means.

For **Subscription**, we select our Azure subscription - this could be Free Trial or Pay-As-You-Go depending on what account type we have. This is just telling Azure which billing account to charge for the resources we're creating.

For **Resource Group**, we can either select an existing one or create a new one. Let's say we create a new resource group called "GL-CosmosDB-RG" to keep everything organized. This way all our Cosmos DB resources stay together and we can easily delete them all at once later.

The **Account Name** is really important. We need to enter a unique name that identifies our Cosmos DB account. Let's say we use something like "glcosmosdb01". This name has to be globally unique across all of Azure because Azure automatically appends ".documents.azure.com" to create the full URI. So our full endpoint would be "glcosmosdb01.documents.azure.com". The name can only contain lowercase letters, numbers, and hyphens, and it has to be between 3 and 44 characters long. If our chosen name isn't available, we just need to try a different one.

For **Location**, we select the region closest to our users. This is important for performance because the closer the data center is to our users, the lower the latency will be. If we're in the US, we might pick East US or West US. If most of our users are in Europe, we'd pick a European region.

The **Capacity mode** setting is where we decide how we want to pay for Cosmos DB. There are two options:

- **Provisioned throughput**: We specify how much capacity we want (measured in Request Units per second or RU/s), and Azure reserves that capacity for us. We pay for this capacity whether we use it all or not.
- **Serverless**: We only pay for the requests we actually make. There's no provisioned capacity - Azure scales automatically based on demand.

For learning purposes, we might select **Serverless** because it's simpler and we won't be charged for unused capacity. In production, we'd choose based on our workload - serverless is great for unpredictable or intermittent workloads, while provisioned throughput is better for consistent, high-volume workloads.

The **Apply Azure Cosmos DB free tier discount** option is really helpful if this is our first Cosmos DB account. With the free tier, we get the first 1000 RU/s and 25 GB of storage for free. We should definitely select "Apply" if it's available.

The **Limit total account throughput** option, if we're using provisioned throughput, lets us set a cap on the maximum throughput that can be provisioned. This is a safety feature to prevent accidentally creating a configuration that costs way more than we expected. We can update or remove this limit later if needed.

Step 3: Configuring Global Distribution

We click on the "Global Distribution" tab to configure how our data is distributed geographically. This is one of the most powerful features of Cosmos DB - it can replicate our data to multiple regions around the world.

For **Geo-Redundancy**, we can enable or disable global distribution. When we enable it, Azure pairs our primary region with a secondary region and keeps them synchronized. For this lab, we'll leave it disabled to keep things simple and avoid extra costs. In production, we'd enable this for disaster recovery and to serve users in different geographic locations.

Multi-region Writes is an advanced feature that lets us write data to multiple regions simultaneously. When disabled, we have one write region and can have multiple read regions.

When enabled, all regions can accept writes, which is great for globally distributed applications but costs more. We'll leave this disabled for the lab.

Availability Zones help improve availability by spreading our data across different data centers within a region. This protects against data center-level failures. We'll leave this disabled for the lab, but in production, this is an important feature for high availability.

Step 4: Optional Configuration Tabs

Azure gives us several more tabs where we can configure additional settings:

The **Networking** tab lets us configure access from virtual networks. We could restrict access so that only resources within a specific VNet can connect to Cosmos DB. For this lab, we'll leave it as default, which allows access from the public internet (but still requires authentication).

The **Backup Policy** tab lets us choose between periodic backups (automated backups at regular intervals) or continuous backup (point-in-time restore capability). The default periodic backup is fine for this lab.

The **Encryption** tab lets us choose between service-managed keys (Azure handles all the encryption keys) or customer-managed keys (we manage the keys ourselves using Azure Key Vault). Service-managed is simpler and fine for most use cases.

The **Tags** tab is for adding metadata labels to our resource. Tags help with organization and cost tracking. For example, we might add tags like "Environment: Lab" or "Project: CosmosDB-Training". These are optional but helpful for managing many resources.

Step 5: Creating the Account

After configuring everything, we select "Review + create". Azure validates our configuration to make sure everything is set up correctly. We should see a green checkbox that says "Validation passed" if everything is okay.

We review all the settings one more time to make sure everything looks right. Then we click "Create" to actually create the Cosmos DB account.

The deployment takes a few minutes. What's happening during this time is pretty complex - Azure is:

- Setting up the database infrastructure
- Allocating storage resources
- Configuring networking and security
- Setting up the management endpoints
- Preparing the account for database and container creation

We can watch the deployment progress in the portal. Once it shows "Your deployment is complete", we click "Go to resource" to navigate to our new Cosmos DB account page.

Step 6: Understanding the Cosmos DB Account Structure

Before we create anything, it's important to understand how Cosmos DB is structured. There's a hierarchy:

1. **Account** (what we just created) - This is the top level that contains everything
2. **Database** - A logical container for collections of data
3. **Container** - Where the actual data is stored (similar to a table in SQL, but more flexible)
4. **Items** - Individual documents stored in the container (JSON objects)

This is different from SQL databases where we have databases containing tables containing rows. In Cosmos DB, we have accounts containing databases containing containers containing items.

Step 7: Creating a Database and Container Using Data Explorer

Now we're ready to create our database and container. We click on "Data Explorer" in the left navigation menu of our Cosmos DB account page. Data Explorer is a built-in tool that lets us manage our databases, containers, and data right from the portal.

At the top of Data Explorer, we see a button that says "New Container". We click on it and might need to scroll right to see the "New Container" window - the Azure portal interface sometimes requires scrolling to see all options.

In the "New Container" pane, we need to configure several settings:

For **Database id**, we enter "ToDoList". This is the name of our new database. Database names have to be between 1 and 255 characters and can't contain forward slashes, backslashes, hash symbols, question marks, or trailing spaces. "ToDoList" is simple and descriptive.

There's a checkbox option that says "Share throughput across containers". This is important to understand. If we check this box, we provision throughput at the database level and all containers in the database share it. If we don't check it, we'd provision throughput for each container individually. Sharing throughput is more cost-effective when we have multiple containers that don't all need maximum capacity at the same time. We should check this box for cost savings.

For **Database throughput**, we need to choose between Autoscale and Manual:

- **Manual**: We set a specific RU/s value and it stays constant unless we manually change it
- **Autoscale**: Azure automatically adjusts the RU/s based on usage, scaling between a minimum and maximum value

For this lab, we select "Manual" because it's simpler to understand. We set it to 400 RU/s, which is the minimum allowed value. This is enough for learning and testing purposes.

It's worth noting that if we created a serverless account, the throughput settings aren't available - serverless accounts don't require us to specify throughput because they scale automatically.

For **Container id**, we enter "Items". This is the name of our container. Container IDs have the same character requirements as database names - they need to be valid identifiers without special characters.

The **Partition key** is one of the most important settings in Cosmos DB. We enter "/category" as the partition key. Let's understand what this means. A partition key determines how Cosmos DB distributes data across multiple physical partitions. When we store a document, Cosmos DB

looks at the value of the partition key field (in this case, "category") and uses it to decide which physical partition to store the document in.

For example, if we have documents with `"category": "personal"` and others with `"category": "work"`, Cosmos DB might store all the "personal" documents on one partition and all the "work" documents on another. This enables Cosmos DB to scale horizontally - it can distribute data across many servers. Choosing a good partition key is critical for performance. We want a key that:

- Has many different values (high cardinality)
- Distributes data evenly across partitions
- Groups related data that we often query together

After filling everything out, we click "OK". Data Explorer creates both the database and the container. We should now see "ToDoList" appear in the left panel, and when we expand it, we'll see our "Items" container inside.

Step 8: Adding Documents to the Container

Now we have a place to store data, but it's empty. Let's add some documents. In Data Explorer, we expand the ToDoList database and then expand the Items container. We'll see several options like "Items", "Scale & Settings", and others.

We click on "Items" and then click the "New Item" button. This opens a document editor on the right side of the screen where we can write JSON.

We add the following JSON document:

```
{
   "id": "1",
   "category": "personal",
   "name": "groceries",
   "description": "Pick up apples and strawberries.",
   "isComplete": false
}
```

Let's break down what each field means:

- `"id": "1"` - This is a required field. Every document in Cosmos DB must have a unique id within its partition. The id combined with the partition key uniquely identifies a document.
- `"category": "personal"` - This is our partition key value. Remember we set "/category" as the partition key, so this field determines which partition this document goes into.
- `"name": "groceries"` - This is just a custom field we're adding. We can add any fields we want.
- `"description": "Pick up apples and strawberries."` - Another custom field with a longer text value.
- `"isComplete": false` - A boolean field tracking whether this task is done.

The beautiful thing about NoSQL databases like Cosmos DB is that we can add any fields we want. There's no schema that forces every document to have the same structure. This is very different from SQL databases where we have to define a table schema first.

We click "Save" and Cosmos DB stores the document. We should see it appear in the items list.

Now we click "New Item" again to create another document. We could create something like:

```
{
  "id": "2",
  "category": "work",
  "name": "presentation",
  "description": "Prepare slides for Monday meeting.",
  "isComplete": true,
  "priority": "high"
}
```

Notice how this document has a different structure - it has a "priority" field that the first document didn't have. This is perfectly fine in Cosmos DB. Each document can have different

fields. The only required field is "id", and we should include the partition key field ("category") so Cosmos DB knows where to store it.

We click "Save" again, and now we have two documents in our container.

Step 9: Querying the Data

At the top of the Items tab in Data Explorer, we see a query box with the default query:

SELECT * FROM c

This is Cosmos DB's SQL-like query language. Let's understand what this query does:

- `SELECT *` means retrieve all fields from the documents
- `FROM c` means from the container (the "c" is just an alias - we could use any letter)

We click the "Execute Query" button (or we might see it as "Apply Filter" depending on the interface) and the query runs. We should see both documents we created displayed as results. The documents are ordered by ID by default.

Now let's try a more interesting query. We click "Edit Filter" and replace the default query with:

ORDER BY c._ts DESC

Wait, this looks incomplete. Actually, the full query is still `SELECT * FROM c`, and we're just adding the ORDER BY clause. So the complete query is:

SELECT * FROM c ORDER BY c._ts DESC

The `c._ts` is a special system property that Cosmos DB automatically adds to every document. It's a timestamp of when the document was last modified, stored as a Unix timestamp (seconds since January 1, 1970). By ordering by `_ts DESC`, we're sorting documents from newest to oldest.

We click "Apply Filter" and the results refresh, now showing documents in order of when they were last modified.

We can try other queries too. For example:

```
SELECT * FROM c WHERE c.category = "personal"
```

This would only return documents where the category is "personal".

Or:

```
SELECT c.name, c.description FROM c WHERE c.isComplete = false
```

This would return only the name and description fields for incomplete tasks.

The query language is very similar to SQL, which makes it easier to learn if we already know SQL. But remember, underneath it's querying JSON documents, not relational tables.

Step 10: Understanding What We've Created

Let's take a step back and understand what we've accomplished and how it all works together.

We created a Cosmos DB account, which is the management boundary. Within that account, we can have multiple databases. We created one database called "ToDoList". Within that database, we created a container called "Items". The container is where our actual data lives.

The partition key ("/category") is doing important work behind the scenes. When we inserted our documents, Cosmos DB looked at the "category" field value and used it to determine physical placement. Documents with the same category value end up on the same logical partition. This matters for:

- **Performance**: Queries that filter by category are efficient because Cosmos DB knows exactly which partition to look in
- **Scalability**: Cosmos DB can spread different category values across different physical servers
- **Transactions**: We can only do multi-document transactions within a single partition

The throughput we provisioned (400 RU/s if using provisioned, or serverless if using that mode) determines how many operations we can perform per second. Every operation (read, write, query) costs some number of Request Units. A simple point read (getting a document by its id and partition key) costs 1 RU. More complex queries cost more RUs.

One of the big differences from SQL databases is the schema flexibility. In SQL, we'd first have to create a table with defined columns. Here, we just start inserting documents with whatever structure we want. This makes it easy to evolve our data model over time - we can add new fields to new documents without having to modify a schema or migrate existing data.

What We Learned and Why This Matters

This lab gave us hands-on experience with NoSQL databases and showed us how different they are from traditional relational databases. The biggest conceptual shift is moving from tables with rows to containers with JSON documents.

The partition key concept is crucial for working with Cosmos DB effectively. Choosing the right partition key can make the difference between a highly performant application and one that doesn't scale well. We want to pick a partition key that:

- Distributes data evenly (not one value that contains 90% of our data)
- Aligns with our query patterns (we can efficiently query within a partition)
- Has high cardinality (many different possible values)

The global distribution capabilities we configured (even though we left them disabled) are one of Cosmos DB's strongest features. In production, we could replicate our data to multiple Azure regions and serve users from the closest region, dramatically reducing latency for global applications.

The schema-less nature of Cosmos DB is both powerful and requires discipline. We can easily add new fields or change document structures, but we need to make sure our application code can handle documents with varying structures. We might have old documents with fewer fields and new documents with additional fields.

The query language being SQL-like is really helpful. If we know SQL, we can start querying Cosmos DB immediately. But we need to remember that the underlying concepts are different - we're not joining tables, we're querying and filtering JSON documents.

If we were building a real application with Cosmos DB, we'd also want to consider:

- **Indexing policy**: By default, Cosmos DB indexes every field, but we can customize this to optimize performance and reduce costs
- **Consistency levels**: Cosmos DB offers five consistency levels (Strong, Bounded Staleness, Session, Consistent Prefix, Eventual) that let us trade consistency for performance
- **Stored procedures and triggers**: We can write JavaScript that runs on the server side
- **Change feed**: We can listen for changes to documents and react to them in real time
- **Time to Live (TTL)**: We can automatically delete old documents after a specified period
- **SDK usage**: Instead of using the portal, we'd typically interact with Cosmos DB through SDKs in our application code

Cosmos DB is particularly well-suited for:

- Applications with globally distributed users
- Applications with unpredictable or spiky workloads (using serverless mode)
- IoT applications that need to ingest large amounts of data
- Gaming applications that need low-latency access to user profiles and game state
- Retail applications with product catalogs that have varying attributes
- Social media applications with user-generated content

This lab gave us the foundation to understand these use cases and start building applications that leverage Cosmos DB's capabilities. The key takeaway is that NoSQL databases like Cosmos DB aren't better or worse than relational databases - they're just different tools designed for different scenarios.

Screen shots:-

Home > Azure Cosmos DB >

Select API option

Which API best suits your workload?

Azure Cosmos DB is a fully managed NoSQL database service for building scalable, high performance applications. Learn more

To start, select the API to create a new account. The API selection cannot be changed after account creation.

Core (SQL) - Recommended

Azure Cosmos DB's core, or native API for working with documents. Supports fast, flexible development with familiar SQL query language and client libraries for .NET, JavaScript, Python, and Java.

Create Learn more

Azure Cosmos DB API for MongoDB

Fully managed database service for apps written for MongoDB. Recommended if you have existing MongoDB workloads that you plan to migrate to Azure Cosmos DB.

Create Learn more

Cassandra

Fully managed Cassandra database service for apps written for Apache Cassandra. Recommended if you have existing Cassandra workloads that you plan to migrate to Azure Cosmos DB.

Create Learn more

Azure Table

Fully managed database service for apps written for Azure Table storage. Recommended if you have existing Azure Table storage workloads that you plan to migrate to Azure Cosmos DB, but do not want to re-write your application to use the SQL API.

Create Learn more

Gremlin (Graph)

Fully managed graph database service using the Gremlin query language, based on Apache TinkerPop project. Recommended for new workloads that need to store relationships between data.

Create Learn more

Home > Azure Cosmos DB > Select API option >

Create Azure Cosmos DB Account - Core (SQL)

Basics | Global Distribution | Networking | Backup Policy | Encryption | Tags | Review + create

Global Distribution

Configure global distribution and regional settings for your account. You can also change these settings after the account is created.

Geo-Redundancy ⓘ ○ Enable ◉ Disable

Multi-region Writes ⓘ ○ Enable ◉ Disable

Home > Azure Cosmos DB > Select API option >

Create Azure Cosmos DB Account - Core (SQL)

Basics | Global Distribution | Networking | Backup Policy | Encryption | Tags | Review + create

Tags are name/value pairs that enable you to categorize resources and view consolidated billing by applying the same tag to multiple resources and resource groups. learn more

Note that if you create tags and then change resource settings on other tabs, your tags will be automatically updated.

Key	Value	Resource Type
cosmosdbseries	cosmosdb	Azure Cosmos DB account
		Azure Cosmos DB account

Home > Microsoft.Azure.CosmosDB-20220807154844 | Overview > samplecosmosdbacc

samplecosmosdbacc | Data Explorer

Azure Cosmos DB account

Search (Cmd+/)

- Overview
- Activity log
- Access control (IAM)
- Tags
- Diagnose and solve problems
- Cost Management
- Quick start
- Notifications
- Data Explorer

Settings

- Features
- Replicate data globally
- Default consistency
- Backup & Restore
- Networking
- CORS
- Dedicated Gateway

New Container | Enable Azure Synapse Link | New Notebook | Connect to GitHub | Connect

SQL API

DATA

- sampledatabase
  - Scale

NOTEBOOKS

Notebooks is currently not available. We are working on it.

Home | Scale

With free tier, you will get the first 1000 RU/s and 25 GB of storage in this account for free. To keep your account free, keep the total RU/s across all resources in the account to 1000 RU/s. Learn more.

Throughput (autoscale)

- (●) Autoscale
- ( ) Manual

Provision maximum RU/s required by this resource. Estimate your required RU/s with capacity calculator

Max RU/s *

1000

Your container throughput will automatically scale from **100 RU/s (10% of max RU/s) - 1000 RU/s** based on usage.

	Min Per Month	Max Per Month
Current Cost	$ 8.76	$ 87.60

(regions: 1, 100 RU/s - 1000 RU/s, $0.00012/RU)

This cost is an estimate and may vary based on the regions where your account is deployed and potential discounts applied to your account

Home > Microsoft.Azure.CosmosDB-20220807154844 | Overview > samplecosmosdbacc

samplecosmosdbacc | Data Explorer
Azure Cosmos DB account

Search (Cmd+/)
Overview
Activity log
Access control (IAM)
Tags
Diagnose and solve problems
Cost Management
Quick start
Notifications
Data Explorer

Settings
Features
Replicate data globally
Default consistency
Backup & Restore
Networking
CORS
Dedicated Gateway
Keys

New Container | Enable Azure Synapse Link | New Notebook | Connect to GitHub

SQL API
DATA
sampledatabase
Scale
NOTEBOOKS
Notebooks is currently not available. We are working on it.

Home | Scale

With free tier, you will get the first 1000 RU/s and 25 GB
the total RU/s across all resources in the account to 100

Throughput (autoscale)
Autoscale
Manual
Provision maximum RU/s required by this resource. Estimate y
Max RU/s *
1000
Your container throughput will automatically scale from **100 R**

	Min Per Month
Current Cost	$ 8.76

(regions: 1, 100 RU/s - 1000 RU/s, $0.00012/RU)
This cost is an estimate and may vary based on the regions w
potential discounts applied to your account
After the first 10 GB of data stored, the max RU/s will be autor

New Container

Your account currently has at least 1 database or container with provisioned RU/s. Billing will apply to this container if the total RU/s in your account exceeds 1000 RU/s. Learn more

* Database id
Create new | Use existing
sampledatabase

* Container id
samplecont

* Indexing
Automatic | Off
All properties in your documents will be indexed by default for flexible and efficient queries. Learn more

* Partition key
For small workloads, the item ID is a suitable choice for the partition key.
/id
Provision dedicated throughput for this container

Unique keys

Home > Microsoft.Azure.CosmosDB-20220807154844 | Overview > samplecosmosdbacc

samplecosmosdbacc | Data Explorer
Azure Cosmos DB account

Search (Cmd+/)
Overview
Activity log
Access control (IAM)
Tags
Diagnose and solve problems
Cost Management
Quick start
Notifications
Data Explorer

Settings
Features
Replicate data globally
Default consistency
Backup & Restore
Networking
CORS
Dedicated Gateway
Keys

New Container | Enable Azure Synapse Link | New Notebook | Connect to GitHub

SQL API
DATA
sampledatabase
Scale
NOTEBOOKS
Notebooks is currently not available. We are working on it.

Home | Scale

With free tier, you will get the first 1000 RU/s and 25 GB
the total RU/s across all resources in the account to 100

Throughput (autoscale)
Autoscale
Manual
Provision maximum RU/s required by this resource. Estimate y
Max RU/s *
1000
Your container throughput will automatically scale from **100 R**

	Min Per Month
Current Cost	$ 8.76

(regions: 1, 100 RU/s - 1000 RU/s, $0.00012/RU)
This cost is an estimate and may vary based on the regions w
potential discounts applied to your account
After the first 10 GB of data stored, the max RU/s will be autor

New Container

samplecontainer

* Indexing
Automatic | Off
All properties in your documents will be indexed by default for flexible and efficient queries. Learn more

* Partition key
For small workloads, the item ID is a suitable choice for the partition key.
/id
Provision dedicated throughput for this container

Unique keys
Add unique key

Analytical store
On | Off
Azure Synapse Link is required for creating an analytical store container. Enable Synapse Link for this Cosmos DB account. Learn more
Enable

Advanced

OK